

Visit 1MY Module 5 Report Draft Content

BMSE3004 Collaborative Development

Student: HO JUN JIE | Student ID: 25WMR09948 | Module 5: Scam Awareness and Quiz

This companion document supplies the missing written content for the AI disclosure, Section A module description, the Message, Constraint and Post-Conditions fields for UC5001 to UC5005, the B3 functional requirements table, Section D implementation discussion and Section E self-reflection. The wording follows the use cases in the submitted report and the current Visit 1MY implementation.

Before Submission

Because ChatGPT has now assisted with drafting and editing report content, update the Nature of Assistance checklist on the AI Usage Disclosure Form to include Editing and Creation, in addition to any already selected categories. Review the text below, revise it into your own voice where needed, and verify every statement against your actual contribution before submission.

Disclosure Statement

ChatGPT was used as a supporting tool during the development and documentation of the Visit 1MY project. I used it to brainstorm module flows, clarify software design concepts, identify possible edge cases, explain and debug selected Flutter and database logic, and help organise and draft parts of the report. I did not accept its output without review. I compared each suggestion with the implemented source code, project requirements and use-case flows, tested the relevant functions, corrected inaccurate or irrelevant content, and revised the wording so that it reflects my own understanding and contribution. The final design decisions, implementation choices, verification of results, interpretation and conclusions were reviewed by me. AI-assisted ideas and text have been disclosed in accordance with the assessment requirements.

SECTION A INTRODUCTION

Module Description

Module 5, Scam Awareness and Quiz, provides tourists with an interactive learning environment for recognising scams and practising safe responses while travelling in Kuala Lumpur. The module presents published safety lessons containing explanations, warning signs, media and recommended actions. Where location permission is available, it can highlight a relevant lesson when the tourist enters a configured hotspot. Tourists can also complete decision-based scenario simulations, receive immediate feedback, attempt timed fraud-awareness quizzes and review their results. Successful learning activities contribute experience points (XP), which are reflected in the tourist's learning profile and can be used to claim available reward vouchers. An administrator-facing Content Management System supports the creation, editing, validation, publication, archiving and deletion of lessons, quizzes and scenarios, as well as reward and performance management. The module integrates the mobile interface, GPS service, Supabase database and server-side procedures to keep learning content, progress, XP and voucher claims consistent.

SECTION B REQUIREMENT STUDY

B2 Use Case Description Completion

UC5001 View Safety Lessons

MESSAGE SECTION

M1: "Location access is unavailable. General safety lessons will be shown."

M2: "Lesson content could not be loaded. Check your internet connection and try again."

M3: "Lesson completed. Your learning progress and XP have been updated."

CONSTRAINT SECTION

C1: Only active, published lessons may be displayed to tourists. A location-based lesson may be highlighted only when valid coordinates are available and the tourist is within the configured hotspot radius.

C2: Lesson completion must be associated with an authenticated tourist. The system must prevent duplicate XP awards according to the configured daily completion rule.

Post-Conditions

1. The selected lesson and its safety guidance have been displayed to the tourist.
2. When the tourist completes the lesson successfully, the completion record is stored and the eligible XP is added to the learning profile.
3. If progress cannot be saved, no confirmed completion is assumed and the tourist is informed of the failure.

UC5002 Play Scenario Simulation

MESSAGE SECTION

M1: "Please select an option to proceed."

M2: "Correct - that is the safest response."

M3: "Scenario completed. Your progress and eligible XP have been recorded."

CONSTRAINT SECTION

C1: Only active scenarios published through the CMS may be played. Each scenario step must contain at least two response options and one valid correct answer.

C2: The tourist cannot proceed without submitting an option. A scenario is recorded as completed only after the final step has been answered correctly, and duplicate XP awards must be prevented by the configured completion rule.

Post-Conditions

1. The tourist has received feedback and an explanation for each submitted response.
2. After the final correct response, the scenario completion is stored in the tourist's learning history and eligible XP is awarded.
3. If saving fails, the simulation result remains visible for review but the system informs the tourist that progress was not recorded.

UC5003 Attempt Fraud Quiz

MESSAGE SECTION

M1: "Quiz questions could not be loaded. Please try again."

M2: "Time is up. The unanswered question has been submitted automatically."

M3: "Quiz completed. View your score, feedback and XP result."

CONSTRAINT SECTION

C1: Only published quiz sets containing at least one valid question may be started. Each question permits one recorded answer and uses its configured time limit.

C2: A score of at least 70 percent is required to pass. XP may be awarded only to an authenticated tourist and only once per quiz within the configured daily reward period.

Post-Conditions

1. All selected answers, unanswered timeouts, total score and elapsed time have been calculated.
2. The quiz attempt is stored, the pass or fail result is determined and eligible XP is added to the tourist's profile.
3. The result screen displays the score, correct and incorrect counts, time taken, feedback and answer-review option even when progress saving fails.

UC5004 View Rewards Profile

MESSAGE SECTION

M1: "No rewards are currently available."

M2: "You do not have enough XP. Complete more activities and try again."

M3: "Reward redeemed successfully. Your voucher code is now available in your profile."

CONSTRAINT SECTION

C1: The tourist must be authenticated to view personal XP, claimed vouchers and redemption status. Only active, unexpired vouchers with available unique codes may be claimed.

C2: A claim may proceed only when the tourist has sufficient spendable XP and has not already claimed the same voucher. XP deduction and assignment of a unique voucher code must be completed as one server-controlled transaction.

Post-Conditions

1. The tourist's current XP balance, reward level, voucher availability and previous claims have been displayed.
2. For an approved claim, the required XP is deducted, a unique voucher code is assigned and the claim is stored in the tourist's profile.
3. If eligibility, stock or server validation fails, no voucher is issued and the tourist receives an appropriate message.

UC5005 Manage Educational Content

MESSAGE SECTION

M1: "Please complete all required fields before saving."

M2: "Content saved successfully as a draft."

M3: "Content published successfully and is now available to tourists."

CONSTRAINT SECTION

C1: Only an authenticated administrator with content-management permission may create, edit, publish, archive or delete educational content.

C2: Mandatory fields and type-specific rules must be valid before saving or publishing. Published quizzes require valid questions and answer indices, while published scenarios require at least two options and one valid correct answer.

Post-Conditions

1. The lesson, quiz or scenario is stored with its selected status: draft, published or archived.
2. Published content is marked active and becomes available in the tourist application; draft or archived content remains unavailable to tourists.

3. If validation or database storage fails, invalid changes are not published and the administrator is shown an actionable error message.

B3 Functional Requirements

The following functional requirements are extracted from the five Module 5 use cases. Each statement describes observable system behaviour and uses a traceable requirement identifier.

USE CASE	REQ. ID	FUNCTIONAL REQUIREMENT STATEMENT
UC5001	FR5001.1	The system shall retrieve the list of active safety lessons from the CMS database.
UC5001	FR5001.2	The system shall request the tourist's location-access status and obtain the current GPS coordinates when permission is granted.
UC5001	FR5001.3	The system shall calculate the distance to configured lesson hotspots and identify the nearest lesson within its permitted radius.
UC5001	FR5001.4	The system shall continue to provide general safety lessons when GPS permission, coordinates or a matching hotspot are unavailable.
UC5001	FR5001.5	The system shall display the selected lesson's text, media, scam warning signs, prevention guidance and recommended actions.
UC5001	FR5001.6	The system shall record lesson completion, update eligible XP and inform the tourist whether progress was saved successfully.
UC5002	FR5002.1	The system shall retrieve active scenario simulations and their ordered steps and response options from the CMS database.
UC5002	FR5002.2	The system shall display the selected scenario situation, optional media and available response options.
UC5002	FR5002.3	The system shall require the tourist to select an option before evaluating a scenario step.
UC5002	FR5002.4	The system shall evaluate the selected response and display immediate corrective or positive feedback with an explanation.
UC5002	FR5002.5	The system shall advance to the next step only after the safe response has been selected and shall support retrying an unsafe response.
UC5002	FR5002.6	The system shall record final scenario completion, update eligible XP and show a completion summary.
UC5003	FR5003.1	The system shall retrieve only published quiz sets that contain valid questions.
UC5003	FR5003.2	The system shall display the question, answer options, progress indicator and configured countdown for each quiz question.
UC5003	FR5003.3	The system shall accept one answer per question and automatically submit an unanswered question when its timer expires.
UC5003	FR5003.4	The system shall compare each submitted answer with the stored correct-answer index and calculate the total score.
UC5003	FR5003.5	The system shall determine whether the tourist passes using the 70 percent passing threshold.
UC5003	FR5003.6	The system shall store the quiz set, score, question total and elapsed time and update eligible XP.
UC5003	FR5003.7	The system shall display the final result, XP outcome, performance breakdown, feedback and an answer-review view.

B3 Functional Requirements Continued

USE CASE	REQ. ID	FUNCTIONAL REQUIREMENT STATEMENT
UC5004	FR5004.1	The system shall retrieve the authenticated tourist's lifetime XP, spendable XP, level, achievements and claimed-voucher count.
UC5004	FR5004.2	The system shall retrieve active reward vouchers, available-code inventory and the tourist's previous claims.
UC5004	FR5004.3	The system shall indicate whether each voucher is locked, ready to redeem, already claimed, used or out of stock.
UC5004	FR5004.4	The system shall display the voucher cost, current XP and projected balance and request confirmation before redemption.
UC5004	FR5004.5	The system shall validate authentication, spendable XP, claim uniqueness and voucher-code availability before issuing a voucher.
UC5004	FR5004.6	The system shall deduct the required XP and assign one unique sponsor code through a server-side claim operation.
UC5004	FR5004.7	The system shall display the issued voucher code and retain it in the tourist's rewards profile for later use.
UC5005	FR5005.1	The system shall restrict educational-content management functions to authorised administrators.
UC5005	FR5005.2	The system shall list existing lessons, quizzes and scenarios with their type, status and update information.
UC5005	FR5005.3	The system shall allow an administrator to create and edit lesson content, including safety guidance, media, XP and optional hotspot information.
UC5005	FR5005.4	The system shall allow an administrator to create and edit quiz sets, questions, options, correct answers, explanations and time limits.
UC5005	FR5005.5	The system shall allow an administrator to create and edit scenario situations, response options, correct answers, feedback and media.
UC5005	FR5005.6	The system shall validate mandatory fields and type-specific content rules before storing any educational content.
UC5005	FR5005.7	The system shall save content as draft, publish it, return it to draft or archive it and shall synchronise its active visibility in the tourist application.
UC5005	FR5005.8	The system shall allow an administrator to delete content after confirmation and shall display the outcome of each save, status-change or deletion operation.

SECTION D IMPLEMENTATION AND CODING

The five segments below were selected because together they cover location-aware learning, interactive decision logic, timed assessment, reward redemption and administrator content publishing. File and line references correspond to the current project source.

D1 Location Aware Lesson Selection

Source: lib/screens/mobile/learning/learning_home_screen.dart, _LearningDashboardData.load

```
final values = await Future.wait([
  repository.getUserProfile(),
  repository.getOverview(),
  repository.getLessons(),
]);
final lessons = values[2] as List<LearningLesson>;
LearningLesson? nearbyLesson;
double? nearbyDistance;
try {
  final status = await locationService.accessStatus();
  if (status == LocationAccessStatus.granted) {
    final position = await locationService.currentPosition().timeout(
      const Duration(seconds: 5),
    );
    for (final lesson in lessons.where(
      (item) =>
        item.isLocationBased &&
        item.latitude != null &&
        item.longitude != null,
    )) {
      final distance = haversineDistanceMeters(
        startLatitude: position.latitude,
        startLongitude: position.longitude,
        endLatitude: lesson.latitude!,
        endLongitude: lesson.longitude!,
      );
      if (distance <= lesson.hotspotRadiusMeters &&
        (nearbyDistance == null || distance < nearbyDistance)) {
        nearbyLesson = lesson;
        nearbyDistance = distance;
      }
    }
  }
} catch (_) {
  // Lessons and quizzes remain available when GPS is unavailable.
}
```

Importance. This segment connects the learning module to the tourist's physical context. Profile data, module statistics and lessons are loaded concurrently, after which the GPS result is used to highlight the nearest eligible hotspot lesson.

Learning points. I learned that asynchronous work can be performed concurrently with Future.wait when the requests do not depend on one another. I also learned to treat location as an enhancement rather than a dependency, so the main learning functions remain usable when permission is denied or positioning fails.

Uniqueness and challenges. The distinctive part is the combination of a per-lesson radius with Haversine distance rather than a simple coordinate comparison. The challenge was balancing relevance, permission handling and response time. A five-second timeout and guarded fallback prevent GPS failure from blocking the entire dashboard.

D2 Scenario Decision and Feedback State

Source: lib/screens/mobile/learning/scenario_gameplay_screen.dart, _submitAnswer and _nextStep

```
void _submitAnswer() {
  if (selectedOption == null) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Please select an option to proceed.')),
    );
    return;
  }
  setState(() {
    showFeedback = true;
  });
}

void _nextStep() {
  if (selectedOption?.isCorrect != true) {
    setState(() {
      selectedOption = null;
    });
  }
}
```

```

        showFeedback = false;
    });
    return;
}
if (currentStepIndex < widget.scenario.steps.length - 1) {
    setState(() {
        currentStepIndex++;
        selectedOption = null;
        showFeedback = false;
    });
} else {
    _showCompletion();
}
}
}

```

Importance. This code forms the state machine for the scenario simulation. It prevents empty submissions, reveals feedback for the selected option, resets unsafe choices for another attempt and moves to completion only after the final safe response.

Learning points. I learned to separate answer submission from step navigation. The separation makes each state transition easier to understand and avoids mixing validation, feedback and progression in one event handler.

Uniqueness and challenges. Unlike a conventional quiz that always advances, this scenario requires the safe response before progression because the purpose is guided practice. The main challenge was keeping selectedOption, showFeedback and currentStepIndex consistent so that old answers do not leak into the next step.

D3 Timed Quiz Scoring and Result Persistence

Source: lib/screens/mobile/learning/quiz_screen.dart, _startQuestion, _submitAnswer and _nextQuestion

```

void _startQuestion() {
    _secondsRemaining = questions[currentQuestionIndex].timeLimitSeconds;
    _stopwatch.start();
    _timer = Timer.periodic(const Duration(seconds: 1), (timer) {
        setState(() {
            if (_secondsRemaining > 0) {
                _secondsRemaining--;
            } else {
                _submitAnswer();
            }
        });
    });
}

void _submitAnswer() {
    if (isAnswered) return;
    _timer?.cancel();
    _stopwatch.stop();
    setState(() {
        isAnswered = true;
        answers[currentQuestionIndex] = selectedOptionIndex ?? -1;
        if (selectedOptionIndex ==
            questions[currentQuestionIndex].correctOptionIndex) {
            correctAnswers++;
        }
    });
}

Future<void> _nextQuestion() async {
    if (currentQuestionIndex < questions.length - 1) {
        setState(() {
            currentQuestionIndex++;
            selectedOptionIndex = null;
            isAnswered = false;
        });
        _startQuestion();
    } else {
        if (_finishing) return;
        setState(() => _finishing = true);
        var saved = true;
        String? saveError;
        var xpAwarded = 0;
        try {
            xpAwarded = await widget.repository.submitQuizAttempt(
                quizSetId: widget.quizSetId,
                score: correctAnswers,
                total: questions.length,
                timeTaken: _stopwatch.elapsed,
            );
        } catch (error) {
            saved = false;
            saveError = error.toString();
        }
        if (!mounted) return;
        Navigator.pushReplacement(

```

```

context,
MaterialPageRoute(
  builder: (_) => QuizResultScreen(
    score: correctAnswers,
    total: questions.length,
    timeTaken: _stopwatch.elapsed,
    progressSaved: saved,
    saveError: saveError,
    xpEarned: xpAwarded,
    questions: questions,
    answers: answers,
  ),
),
);
}
}
}

```

Importance. This segment controls the assessment lifecycle. Every question receives its own countdown, an unanswered timeout is recorded as -1, duplicate submission is blocked, correct answers are counted and the completed attempt is sent to the repository with elapsed time.

Learning points. I learned that timers require explicit lifecycle control. Cancelling the timer before changing state avoids multiple submissions, while a separate stopwatch measures the total attempt duration. I also learned that the result should still be shown when saving fails, with a clear persistence-status message.

Uniqueness and challenges. The segment combines real-time UI state with durable backend progress. The difficult part was preventing race conditions between a user's tap and the timer reaching zero. The `isAnswered` guard and timer cancellation make the submission idempotent at the screen level, while server-side logic determines the final XP award.

D4 Reward Eligibility and Server Controlled Claim

Source: `lib/data/learning_repository.dart`, `getVouchers` and `claimVoucher`

```

Future<List<RewardVoucher>> getVouchers() async {
  final userId = client.auth.currentUser?.id;
  final profile = userId != null ? await getUserProfile() : null;

  final response = await client
    .from('reward_vouchers')
    .select('*', user_claimed_vouchers(user_id, full_promo_code, used_at'))
    .eq('is_active', true);

  final inventoryByVoucher = <String, int>{};
  try {
    final inventory = await client.rpc('get_reward_inventory');
    for (final row in inventory as List) {
      inventoryByVoucher[row['voucher_id'].toString()] =
        (row['available_codes'] as num?)?.toInt() ?? 0;
    }
  } on PostgreSQLException {
    // The inventory migration may not have been applied yet.
  }

  final List<dynamic> data = response as List<dynamic>;

  return data.map((v) {
    final List<dynamic> claimedData = v['user_claimed_vouchers'] ?? [];
    final ownClaims = claimedData
      .where((claim) => claim['user_id'] == userId)
      .toList();
    final bool isClaimed = ownClaims.isNotEmpty;
    final String? fullCode = isClaimed
      ? ownClaims.first['full_promo_code']
      : null;
    final int requiredXp = v['required_xp'] ?? 0;

    return RewardVoucher(
      id: v['id'].toString(),
      referenceCode: v['voucher_code'] ?? '',
      partnerName: v['partner_name'],
      title: v['title'],
      discountAmount: v['discount_amount'],
      expiryDate: DateTime.parse(v['valid_until']),
      requiredXp: requiredXp,
      isUnlocked: profile != null && profile.spendableXp >= requiredXp,
      isClaimed: isClaimed,
      isUsed: isClaimed && ownClaims.first['used_at'] != null,
      promoCode: fullCode,
      availableCodes: inventoryByVoucher[v['id'].toString()] ?? 0,
    );
  }).toList();
}

```

```

Future<String> claimVoucher(String voucherId) async {
  final userId = client.auth.currentUser?.id;
  if (userId == null) throw Exception("User not authenticated");

  final result = await client.rpc(
    'claim_reward_voucher',
    params: {'target_voucher_id': voucherId},
  );
  if (result is String) return result;
  return result.toString();
}

```

Importance. Rewards have financial-like state because XP must be deducted and a limited unique code must be assigned. This segment combines the tourist's balance, existing claims and live voucher inventory to determine display state, then delegates the actual claim to a protected server-side procedure.

Learning points. I learned that client-side checks are useful for feedback but cannot be the final authority. Authentication, sufficient XP, stock availability and claim uniqueness must be verified on the server because client state can become stale or be manipulated.

Uniqueness and challenges. The unique aspect is the separation between lifetime XP and spendable XP together with limited sponsor-code inventory. The challenge was avoiding double claims and overselling the final code. A database RPC can perform validation, XP deduction and code assignment atomically, which is safer than several independent client updates.

D5 Status Aware CMS Persistence

Source: lib/data/awareness_admin_repository.dart, saveLesson and setContentStatus

```

Future<void> saveLesson(AdminLessonDraft draft) async {
  final data = {
    'title': draft.title,
    'category': draft.category,
    'difficulty': draft.difficulty,
    'read_time': draft.readTime,
    'content': draft.content,
    'red_flags': draft.redFlags,
    'what_to_do': draft.whatToDo,
    'xp_reward': draft.xpReward,
    'status': draft.status,
    'is_active': draft.status == 'published',
    'image_url': _nullable(draft.imageUrl),
    'hotspot_label': _nullable(draft.hotspotLabel),
    'latitude': draft.latitude,
    'longitude': draft.longitude,
    'hotspot_radius_meters': draft.hotspotRadiusMeters,
    'is_location_based': draft.isLocationBased,
    'published_at': draft.status == 'published'
      ? DateTime.now().toIso8601String()
      : null,
    'updated_at': DateTime.now().toIso8601String(),
  };
  if (draft.id == null) {
    await client.from('learning_lessons').insert(data);
  } else {
    await client.from('learning_lessons').update(data).eq('id', draft.id!);
  }
}

Future<void> setContentStatus(
  AwarenessContentSummary item,
  String status,
) async {
  if (item.type == AwarenessContentType.quiz) {
    await client.rpc(
      'set_quiz_set_status',
      params: {'target_quiz_set_id': item.id, 'target_status': status},
    );
    return;
  }
  final table = switch (item.type) {
    AwarenessContentType.lesson => 'learning_lessons',
    AwarenessContentType.quiz => throw StateError('Handled above'),
    AwarenessContentType.scenario => 'scenarios',
  };
  await client
    .from(table)
    .update({
      'status': status,
      'is_active': status == 'published',
      'updated_at': DateTime.now().toIso8601String(),
      if (status == 'published')
        'published_at': DateTime.now().toIso8601String(),
    })
}

```

```
    .eq('id', item.id);  
}
```

Importance. This repository code is the boundary between the administrator editor and the CMS database. It maps a validated draft to database fields, chooses insert or update based on the presence of an identifier and keeps publication status synchronised with tourist visibility.

Learning points. I learned the value of keeping persistence logic outside the widget layer. The editor manages validation and user interaction, while the repository owns table names, payloads, timestamps and status mapping. This makes both parts easier to maintain and test.

Uniqueness and challenges. The important design detail is that published and active are updated together. Without this mapping, content could appear in the tourist app while still marked as a draft, or disappear despite being published. The challenge was supporting different content lifecycles consistently while preserving optional values and accurate audit timestamps.

SECTION E SELF REFLECTION

E1 Most Significant Lesson

The most significant lesson I learned was that software correctness depends on the complete flow between the interface, application state and database, rather than on whether one screen appears to work. This was meaningful because Module 5 handles progress, XP and limited voucher codes, where an incorrect update can affect both user trust and data consistency. A specific experience was implementing the voucher-claim flow. At first, checking the XP balance in the interface seemed sufficient. However, I realised that the displayed balance and code inventory could change before the tourist confirmed the claim. The final design therefore uses the interface to explain the cost and request confirmation, but a server-side procedure performs the authoritative checks, XP deduction and unique-code assignment. This experience changed how I evaluate a feature: I now trace the whole transaction, identify where the source of truth belongs and consider failure states before treating the feature as complete.

E2 Perception of the Development Process

This exercise changed my view of software development from a mostly linear process into an iterative cycle of requirements, design, implementation, testing and refinement. Before this project, I tended to think that a complete use-case description could be converted directly into screens and code. During implementation, I found that practical constraints repeatedly required the design to be revisited. For example, location-aware lessons needed a timeout and a fallback so that denied GPS permission would not block general learning content. The scenario completion overlay also had to be adjusted after testing showed that a long explanation could overflow on a smaller viewport. These issues were not visible in the initial diagrams, but they affected usability and reliability. I now understand that diagrams and prototypes are starting hypotheses. They must be validated against actual data, device sizes, permissions, errors and user behaviour, and the lessons from testing should be fed back into the requirements and design documentation.

E3 Collaborative Software Development

The exercise strengthened my understanding that collaboration is not simply dividing the application into separate modules. The main benefit of teamwork was parallel progress: different members could concentrate on their assigned functions while sharing common navigation, user accounts, database tables and design conventions. This allowed the project to cover a wider scope than one person could complete in the same period. At the same time, shared dependencies created integration challenges. A change to a model, route, database column or common widget could affect screens owned by other members. This made communication about interface contracts and schema changes as important as writing the feature itself. I learned to inspect related modules before changing shared code, keep changes focused, test integration points and explain assumptions clearly when coordinating with teammates. The project showed me that effective collaboration requires both individual ownership and collective responsibility for the behaviour of the integrated system.

E4 Performance Strengths and Areas for Improvement

My main strengths were structured problem solving, persistence during debugging and attention to the tourist's experience. I broke Module 5 into smaller flows for lessons, simulations, quizzes, progress and rewards, which helped me connect each interface action to a repository operation and user message. I also considered recoverable failure states, such as unavailable GPS, failed progress saving and exhausted voucher inventory. These strengths helped the team by reducing unclear behaviour at integration time and by making the module usable beyond the ideal success path. My main area for improvement is to introduce formal traceability and automated widget testing earlier. Some layout and integration problems were found only after the screens were already developed. I could also communicate database assumptions and reusable interface changes earlier so that teammates have more time to adapt their modules. Improving these areas would reduce rework, shorten integration cycles and give the team clearer evidence that each use case has been implemented completely.

E5 Actions I Would Take Differently

If I repeated the exercise, I would begin by creating a requirement-to-test matrix for every use case and agree on shared models, route names and database contracts with the team before substantial implementation. I would develop in smaller increments, integrate each completed flow earlier and schedule regular demonstrations on both narrow mobile and wide desktop layouts. I would also add widget tests for long content, large text and loading or error states, together with repository tests for duplicate XP, quiz-attempt recording and voucher-claim edge cases. For collaboration, I would document schema changes and shared-component decisions immediately and use shorter branches or commits so that reviews and merges remain manageable. Finally, I would reserve time after implementation to update diagrams and report statements according to the final behaviour. These actions would make my learning more systematic, expose misunderstandings earlier and produce a more consistent and verifiable project outcome.